

Table of Contents

GCC Installation and Version Check.....	3
Task 1: Compile hello.c, verify exit status.....	3
Summary:.....	3
Task 2: Modify program to return 1.....	4
Summary.....	4
Task 3: Program that accepts and displays user input.....	4
Summary.....	4
Task 4: Run all four compilation stages separately.....	5
Summary.....	7
Program with formatted output.....	7
Knowledge Check Questions.....	8

Table of Figures

Figure 1:GCC Installation and Version Check.....	3
Figure 2: Compile hello.c.....	3
Figure 3: Modify Program.....	4
Figure 4: Program that accepts and display user input.....	4
Figure 5: Preprocessing stage.....	5
Figure 6: Compilation stage.....	6
Figure 7: Assembly stage.....	6
Figure 8: Linking stage.....	7
Figure 9: Check all File Exists.....	7
Figure 10: program with formatted output.....	7

GCC Installation and Version Check

```
ashim@Ubuntu:~$ mkdir os_lab2
ashim@Ubuntu:~$ cd os_lab2
ashim@Ubuntu:~/os_lab2$ sudo apt install build-essential
[sudo: authenticate] Password:
build-essential is already the newest version (12.12ubuntu2.26.04.2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 5
ashim@Ubuntu:~/os_lab2$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

ashim@Ubuntu:~/os_lab2$
```

Figure 1: GCC Installation and Version Check

Task 1: Compile hello.c, verify exit status

```
ashim@Ubuntu:~/os_lab2$ nano hello.c
ashim@Ubuntu:~/os_lab2$ gcc hello.c -o hello
ashim@Ubuntu:~/os_lab2$ ./hello
Hello world!
ashim@Ubuntu:~/os_lab2$ echo $?
0
ashim@Ubuntu:~/os_lab2$
```

Figure 2: Compile hello.c

Summary:

Compiled hello.c successfully with gcc, ran the executable, and confirmed the program returned exit status 0, indicating successful execution.

Task 2: Modify program to return 1

```
ashim@Ubuntu:~/os_lab2$ nano hello.c
ashim@Ubuntu:~/os_lab2$ gcc hello.c -o hello
ashim@Ubuntu:~/os_lab2$ ./hello
Hello world!
ashim@Ubuntu:~/os_lab2$ echo $?
1
ashim@Ubuntu:~/os_lab2$
```

Figure 3: Modify Program

Summary

Modified the return value to 1 and recompiled; echo \$? confirmed the exit status changed from 0 to 1, showing the OS captures the value returned by main().

Task 3: Program that accepts and displays user input

```
ashim@Ubuntu:~/os_lab2$ nano input.c
ashim@Ubuntu:~/os_lab2$ gcc input.c -o input
ashim@Ubuntu:~/os_lab2$ ./input
Enter a number: 12
You entered: 12
ashim@Ubuntu:~/os_lab2$
```

Figure 4: Program that accepts and display user input

Summary

Wrote a C program that declares an integer variable, reads user input using scanf(), and displays it back with printf(), demonstrating basic I/O handling in C.

Task 4: Run all four compilation stages separately

Step 1 - Preprocessing

```
ashim@Ubuntu:~/os_lab2$ gcc -E hello.c -o hello.i
ashim@Ubuntu:~/os_lab2$ cat hello.i | less
```

```
# 0 "hello.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "hello.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
# 563 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 564 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 29 "/usr/include/stdio.h" 2 3 4
```

Figure 5: Preprocessing stage

Step 2 - Compilation to Assembly

```
ashim@Ubuntu:~/os_lab2$ gcc -S hello.i -o hello.s
ashim@Ubuntu:~/os_lab2$ cst hello.s
Command 'cst' not found, but can be installed with:
sudo apt install imx-code-signing-tool
ashim@Ubuntu:~/os_lab2$ cat hello.s
.file "hello.c"
.text
.section      .rodata
.LC0:
.string "Hello world!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
endbr64
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
leaq .LC0(%rip), %rax
movq %rax, %rdi
call puts@PLT
movl $1, %eax
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0"
.section      .note.GNU-stack,"",@progbits
.section      .note.gnu.property,"a"
.align 8
.long 1f - 0f
.long 4f - 1f
.long 5
0:
.string "GNU"
1:
.align 8
.long 0xc0000002
.long 3f - 2f
```

Figure 6: Compilation stage

Step 3 - Assembly to Object File

```
ashim@Ubuntu:~/os_lab2$ gcc -c hello.s -o hello.o
ashim@Ubuntu:~/os_lab2$ file hello.o
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
ashim@Ubuntu:~/os_lab2$
```

Figure 7: Assembly stage

Step 4: Linking

```
ashim@Ubuntu:~/os_lab2$ gcc hello.o -o hello
ashim@Ubuntu:~/os_lab2$ ./hello
Hello world!
ashim@Ubuntu:~/os_lab2$
```

Figure 8: Linking stage

Step 5 - Check all File Exists

```
ashim@Ubuntu:~/os_lab2$ ls -la hello.i hello.s hello.o hello
-rwxrwxr-x 1 ashim ashim 15952 Sep 16 15:36 hello
-rw-rw-r-- 1 ashim ashim 21427 Sep 16 15:15 hello.i
-rw-rw-r-- 1 ashim ashim 1496 Sep 16 15:26 hello.o
-rw-rw-r-- 1 ashim ashim 660 Sep 16 15:24 hello.s
ashim@Ubuntu:~/os_lab2$
```

Figure 9: Check all File Exists

Summary

Ran each compilation stage manually – preprocessing (.i), compiling to assembly (.s), assembling to object code (.o), and linking to produce the final executable – confirming how source code is progressively transformed into a machine-executable program.

Program with formatted output

```
ashim@Ubuntu:~/os_lab2$ nano formatted.c
ashim@Ubuntu:~/os_lab2$ gcc formatted.c -o formatted
ashim@Ubuntu:~/os_lab2$ ./formatted
Enter your name: Ashim
Enter your age: 23
Enter your height (m): 1.8

--User Info--
Name: Ashim
Age: 23 years
Height: 1.80 m
ashim@Ubuntu:~/os_lab2$
```

Figure 10: program with formatted output

Knowledge Check Questions

What is the purpose of the `#include` directive in C programming? Provide an example of its usage.

It tells the preprocessor to insert the contents of a header file (like function declarations) before compilation. Example: `#include <stdio.h>` gives access to `printf()/scanf()`.

Explain the difference between the four stages of compilation. What file extension is produced at each stage?

Preprocessing produces `.i` (expands macros/headers); Compiling produces `.s` (generates assembly code); Assembling produces `.o` (creates object/machine code); Linking produces the final executable (combines object files/libraries).

What does a return value of 0 from the `main()` function indicate to the operating system? What about a non-zero return value?

0 tells the OS the program executed successfully; a non-zero value indicates an error occurred, and the specific number can be used to signal what kind of error.

What GCC command would you use to:

- 1) Only preprocess a file without compiling?
- 2) Generate assembly code but not assemble it?
- 3) Create an object file without linking?

Only preprocess a file without compiling => `gcc -E file.c -o file.i`

Generate assembly code but not assemble it => `gcc -S file.c -o file.s`

Create an object file without linking => `gcc -c file.c -o file.o`

Explain why C is called a "compiled language" and how this differs from interpreted languages.

C source code is fully translated into machine code before execution (via preprocessing, compiling, assembling, linking), producing a standalone executable that runs directly on the hardware. Interpreted languages, by contrast, are translated and executed line-by-line at runtime by an interpreter, without producing a separate machine-code executable.